OBJECTIVE-C · ПРЕЗЕНТАЦИЯ 3 ИЗ 3

Objective-C: модель исполнения

Конкурентность · ООП, протоколы, ФП · специфические конструкции · среда выполнения · управление памятью

План презентации

ЧАСТЬ 1

Основы

- Общие сведения: история, особенности, версия
- Как запустить: Docker, онлайн, локально
- Данные и переменные, владение
- Функции, рекурсия, замыкания (блоки)
- Условия, циклы, итераторы

ЧАСТЬ 2

Надёжность и структура

- Исключения, NSError, assert
- Префиксы, заголовки, модули, библиотеки
- Обобщённые типы
- Коллекции Foundation
- Ввод-вывод: консоль и файлы

ЧАСТЬ 3

Модель исполнения

- Конкурентность: NSThread, GCD, NSOperation
- ООП, протоколы, категории, ФП
- Специфические конструкции: сообщения, KVC/KVO
- Среда выполнения: рантайм, objc_msgSend
- Управление памятью: MRC, ARC

Средства конкурентности

Уровень	Средство	Когда использовать
низкий	`pthread` (C)	совместимость с C-кодом
	`NSThread`	отдельный поток с объектным API
синхронизация	`@synchronized`, `NSLock`, `NSRecursiveLock`, `NSCondition`	защита общих данных
	`atomic`-свойства	атомарный get/set одного свойства (но не составных операций!)
высокий	**GCD** (`libdispatch`): очереди, группы, семафоры	основной инструмент: задачи вместо потоков
	`NSOperationQueue`	задачи с зависимостями, отменой, приоритетами

> GCD входит в macOS; на Linux и Windows тот же API даёт `libdispatch` (swift-corelibs).

Гонка данных и синхронизация

01_threads.m

```
@interface Counter : NSObject
@property (nonatomic) long unsafeValue;            // nonatomic — без защиты
@property (nonatomic) long syncValue;
@property (nonatomic) long lockValue;
@end
@implementation Counter {
    NSLock *_lock;
}
- (instancetype)init { if ((self = [super init])) _lock = [NSLock new]; return self; }

- (void)work {
    for (int i = 0; i < kIters; i++) {
        self.unsafeValue++;                        // read-modify-write: гонка!
        @synchronized (self) { self.syncValue++; }    // монитор на объекте
        [_lock lock];  self.lockValue++;  [_lock unlock];
    }
}
@end
```

NSThread: запускаем 4 потока

01_threads.m

```
Counter *c = [Counter new];
NSMutableArray<NSThread *> *threads = [NSMutableArray array];
for (int t = 0; t < kThreads; t++) {
    NSThread *th = [[NSThread alloc] initWithTarget:c selector:@selector(work) object:nil];
    [threads addObject:th];
    [th start];
}
for (NSThread *th in threads)                  // простое ожидание завершения
    while (!th.isFinished) [NSThread sleepForTimeInterval:0.01];

NSLog(@"expected %d", kThreads * kIters);
NSLog(@"unsafe=%ld sync=%ld lock=%ld", c.unsafeValue, c.syncValue, c.lockValue);
```

```
$ ./01_threads
expected 800000
unsafe=795155 sync=800000 lock=800000
```

> Без синхронизации счётчик теряет инкременты — и каждый запуск даёт разное число.

GCD: задачи и группы

02_gcd.m

```m
dispatch_queue_t global = dispatch_get_global_queue(DISPATCH_QUEUE_PRIORITY_DEFAULT, 0);
dispatch_group_t group = dispatch_group_create();
__block long a = 0, b = 0;

dispatch_group_async(group, global, ^{ for (long i = 1; i <= 1000; i++) a += i; });
dispatch_group_async(group, global, ^{ for (long i = 1; i <= 2000; i++) b += i; });

dispatch_group_wait(group, DISPATCH_TIME_FOREVER);     // join
NSLog(@"a=%ld b=%ld", a, b);
```

```
$ ./02_gcd
a=500500 b=2001000
```

- Задача = блок, очередь = кто и как выполняет
- Группа = ожидание набора задач (join)

NSOperationQueue: зависимости

03_operations.m

```m
NSOperationQueue *queue = [NSOperationQueue new];
queue.maxConcurrentOperationCount = 2;                  // ограничение параллелизма

NSBlockOperation *download = [NSBlockOperation blockOperationWithBlock:^{
    NSLog(@"download");
}];
NSBlockOperation *parse = [NSBlockOperation blockOperationWithBlock:^{
    NSLog(@"parse");
}];
NSBlockOperation *save = [NSBlockOperation blockOperationWithBlock:^{
    NSLog(@"save");
}];
[parse addDependency:download];                         // граф зависимостей
[save addDependency:parse];

[queue addOperations:@[save, parse, download] waitUntilFinished:YES];
NSLog(@"pipeline done");
```

```
$ ./03_operations
download
parse
save
pipeline done
```

ООП: интерфейс класса

04_oop.m

```m
@interface Animal : NSObject                        // интерфейс (обычно в .h)
@property (nonatomic, copy, readonly) NSString *name;
@property (nonatomic) NSUInteger age;
+ (instancetype)animalWithName:(NSString *)name;   // фабричный метод класса
- (instancetype)initWithName:(NSString *)name;     // назначенный инициализатор
- (NSString *)sound;
- (NSString *)speak;
@end
```

- Только одиночное наследование; корень — `NSObject`
- `@property` генерирует ivar `_name`, геттер и сеттер
- `instancetype` — «тип того класса, у которого вызвали»
- Создание объекта в два шага: `alloc` (память) + `init` (инициализация)

Наследование и полиморфизм

04_oop.m

```
@interface Dog : Animal
@end
@implementation Dog
- (NSString *)sound { return @"Woof"; }           // переопределение
@end

@interface Puppy : Dog
@end
@implementation Puppy
- (NSString *)sound {
    return [[super sound] lowercaseString];       // вызов реализации родителя
}
@end

@interface Cat : Animal
@end
@implementation Cat
- (NSString *)sound { return @"Meow"; }
@end
```

Полиморфизм и интроспекция

04_oop.m

```
NSArray<Animal *> *zoo = @[[Dog animalWithName:@"Rex"],
                           [Cat animalWithName:@"Tom"],
                           [Puppy animalWithName:@"Bim"]];
for (Animal *a in zoo) NSLog(@"%@", [a speak]);    // полиморфизм: позднее связывание

Animal *bim = zoo[2];
bim.age = 1;                          // точечный синтаксис = [bim setAge:1]
[bim setAge:[bim age] + 1];           // то же самое сообщениями
NSLog(@"%@", bim);
NSLog(@"isKindOf Dog: %d, isMemberOf Dog: %d, superclass: %@",
      [bim isKindOfClass:[Dog class]], [bim isMemberOfClass:[Dog class]],
      [[bim class] superclass]);
```

```
$ ./04_oop
Rex the Dog says Woof
Tom the Cat says Meow
Bim the Puppy says woof
<Puppy Bim, 2 y>
isKindOf Dog: 1, isMemberOf Dog: 0, superclass: Dog
```

Протоколы: контракт без реализации

05_protocols.m

```
@protocol Shape <NSObject>                  // протокол наследует протокол NSObject
@required
- (double)area;
@optional
- (NSString *)name;
@end
```

05_protocols.m

```
double totalArea(NSArray<id<Shape>> *shapes) {     // работаем с протоколом, не с классом
    double sum = 0;
    for (id<Shape> s in shapes) sum += [s area];
    return sum;
}
```

- `@required` — обязательные, `@optional` — необязательные методы
- `id<Shape>` — «любой объект, поддерживающий протокол»

Категории: методы для чужих классов

06_categories.m

```objc
@interface NSString (Demo)                    // категория Demo для системного NSString
- (NSString *)reversed;
- (BOOL)isPalindrome;
@end

@implementation NSString (Demo)
- (NSString *)reversed {
    NSMutableString *r = [NSMutableString stringWithCapacity:self.length];
    for (NSInteger i = (NSInteger)self.length - 1; i >= 0; i--)
        [r appendFormat:@"%C", [self characterAtIndex:(NSUInteger)i]];
    return r;
}
- (BOOL)isPalindrome {
    NSString *clean = [[self lowercaseString] stringByReplacingOccurrencesOfString:@" " withString:@""];
    return [clean isEqualToString:[clean reversed]];
}
@end
```

```
$ ./06_categories
desserts | 1 | 0
```

Функциональный стиль: map / filter / reduce

07_functional.m

```
@interface NSArray<ObjectType> (FP)
- (NSArray *)map:(id (^)(ObjectType x))f;
- (NSArray<ObjectType> *)filter:(BOOL (^)(ObjectType x))pred;
- (id)reduce:(id)initial with:(id (^)(id acc, ObjectType x))f;
@end

@implementation NSArray (FP)
- (NSArray *)map:(id (^)(id))f {
    NSMutableArray *out = [NSMutableArray arrayWithCapacity:self.count];
    for (id x in self) [out addObject:f(x)];
    return [out copy];                          // наружу — неизменяемый результат
}
- (NSArray *)filter:(BOOL (^)(id))pred {
    return [self objectsAtIndexes:[self indexesOfObjectsPassingTest:
            ^BOOL(id x, NSUInteger i, BOOL *stop) { return pred(x); }]];
}
- (id)reduce:(id)acc with:(id (^)(id, id))f {
    for (id x in self) acc = f(acc, x);
    return acc;
}
@end
```

Конвейер из функций

07_functional.m

```
NSArray<NSNumber *> *nums = @[@1, @2, @3, @4, @5, @6];

NSArray *squares = [nums map:^id(NSNumber *x) { return @(x.intValue * x.intValue); }];
NSArray *evens   = [nums filter:^BOOL(NSNumber *x) { return x.intValue % 2 == 0; }];
NSNumber *sum    = [nums reduce:@0 with:^id(NSNumber *acc, NSNumber *x) {
    return @(acc.intValue + x.intValue);
}];
NSLog(@"squares=%@ evens=%@ sum=%@", [squares componentsJoinedByString:@","],
      [evens componentsJoinedByString:@","], sum);

Fn inc = adder(1), dbl = ^id(NSNumber *x) { return @(x.intValue * 2); };
Fn incThenDouble = compose(inc, dbl);
NSLog(@"(5+1)*2 = %@", incThenDouble(@5));

NSArray *pipeline = [[[nums filter:^BOOL(NSNumber *x) { return x.intValue > 2; }]
                       map:incThenDouble]
                      map:^id(NSNumber *x) { return x.stringValue; }];
NSLog(@"pipeline: %@", [pipeline componentsJoinedByString:@" "]);
```

```
$ ./07_functional
squares=1,4,9,16,25,36 evens=2,4,6 sum=21
(5+1)*2 = 12
pipeline: 8 10 12 14
```

Отправка сообщения объекту nil

08_messaging.m

```
Robot *nobody = nil;
NSString *r = [nobody greet:@"you"];    // сообщение nil — не ошибка: вернёт nil/0
NSInteger b = nobody.battery;
NSLog(@"nil result: %@, %ld", r, (long)b);
```

```
$ ./08_messaging
nil result: (null), 0
```

Цепочки вида `[[[a b] c] d]` не требуют проверок на nil на каждом шаге.

Селекторы и динамическая типизация

08_messaging.m

```
        Robot *robo = [Robot new];
        robo.name = @"R2";
        SEL sel = NSSelectorFromString(@"greet:");            // имя метода как данные
        if ([robo respondsToSelector:sel]) {
#pragma clang diagnostic push
#pragma clang diagnostic ignored "-Warc-performSelector-leaks"   // ARC не знает, что вернёт sel
            NSString *res = [robo performSelector:sel withObject:@"C3PO"];
#pragma clang diagnostic pop
            NSLog(@"%@", res);
        }
        id any = robo;                                        // динамическая типизация:
        NSLog(@"%@", [any greet:@"id"]);                      // проверка — в рантайме
        @try { [any count]; } @catch (NSException *e) { NSLog(@"%@", e.reason); }
```

```
$ ./08_messaging
R2 greets C3PO
R2 greets id
Robot(instance) does not recognize count
```

KVO: наблюдение за свойствами

08_messaging.m

```
@interface Monitor : NSObject
@end
@implementation Monitor
- (void)observeValueForKeyPath:(NSString *)keyPath ofObject:(id)obj
                        change:(NSDictionary *)change context:(void *)ctx {
    NSLog(@"KVO: %@ %@ -> %@", keyPath, change[NSKeyValueChangeOldKey],
          change[NSKeyValueChangeNewKey]);
}
@end
```

08_messaging.m

```
Monitor *mon = [Monitor new];
[robo addObserver:mon forKeyPath:@"battery"
          options:NSKeyValueObservingOptionOld | NSKeyValueObservingOptionNew
          context:NULL];
robo.battery = 50;                                    // наблюдатель узнает сам
robo.battery = 10;
[robo removeObserver:mon forKeyPath:@"battery"];
```

```
$ ./08_messaging
KVO: battery 87 -> 50
KVO: battery 50 -> 10
```

Директивы @ — краткий справочник

Директива	Назначение
`@interface`, `@implementation`, `@end`	объявление и реализация класса
`@property`, `@synthesize`, `@dynamic`	свойства, генерация / отказ от генерации методов
`@protocol`, `@optional`, `@required`	протоколы
`@class`	предварительное объявление класса
`@selector(...)`, `@protocol(...)`, `@encode(...)`	селектор, объект-протокол, строка типа
`@try`, `@catch`, `@finally`, `@throw`	исключения
`@synchronized`, `@autoreleasepool`	монитор, пул автоосвобождения
`@"…"`, `@42`, `@[…]`, `@{…}`, `@(expr)`	литералы объектов
`@import`, `@available`	модули, проверка версии ОС

Модель исполнения: машинный код и рантайм



Рантайм — это библиотека

- `libobjc`: Apple objc4 или GNUstep libobjc2
- классы, селекторы, `objc_msgSend`
- ARC: `objc_retain`, `objc_release`
- интроспекция, добавление методов

Чего нет

- байт-кода и JIT
- сборщика мусора (с 2012 г. устарел)
- изоляции и проверок как в JVM
- переносимости бинарников

Что даёт

- скорость C
- динамику Smalltalk
- прямой вызов кода на C/C++
- ABI с Swift (на Apple)

Отправка сообщения = вызов objc_msgSend

11_compiled.m

```
NSUInteger callIt(NSString *s) {
    return [s length];       // -> objc_msgSend(s, @selector(length))
}
```

LLVM IR (-O1)

```
define i64 @callIt(ptr noundef %0) {
  %2 = tail call i64 @objc_msgSend(ptr noundef %0,
         ptr @".objc_selector_length_Q16\010:8")
  ret i64 %2
}
```

x86-64 (objdump -dr)

```
callIt:
  lea   rsi, [rip + .objc_selector_length]
  jmp   objc_msgSend      ; хвостовой вызов
```

`[s length]` → `objc_msgSend(s, @selector(length))`: селектор во втором регистре, дальше рантайм ищет реализацию. Селектор GNUstep типизирован: `length` + сигнатура `Q16@0:8`.

Как objc_msgSend находит метод

nil?
→ вернуть 0

→

кэш
быстрый путь

→

методы класса
медленный путь

→

суперклассы
вверх по иерархии

`resolveInstanceMethod:`
добавить метод на лету

→

`forwardingTargetForSelector:`
отдать другому объекту

→

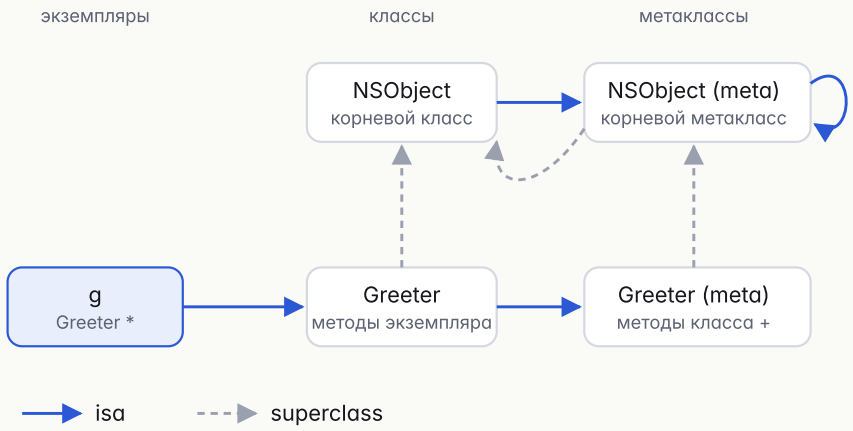
`forwardInvocation:`
полная пересылка

→

`doesNotRecognizeSelector:`
исключение

Найденная реализация кладётся в кэш класса — повторный вызов почти так же быстр, как вызов C-функции.

Объектная модель: isa, класс, метакласс



09_runtime.m

```
Greeter *g = [Greeter new];
Class cls  = object_getClass(g);                    // isa объекта -> класс
Class meta = object_getClass(cls);                  // isa класса -> метакласс
NSLog(@"%s -> %s (meta: %d) -> super %s", class_getName(cls), class_getName(meta),
      class_isMetaClass(meta), class_getName(class_getSuperclass(cls)));
```

```
$ ./09_runtime
Greeter -> Greeter (meta: 1) -> super NSObject
```

Эволюция управления памятью

MRC
1980-е → `retain` / `release` / `autorelease` вручную

→

Сборщик мусора
2007, Mac OS X 10.5; устарел в 10.8 (2012)

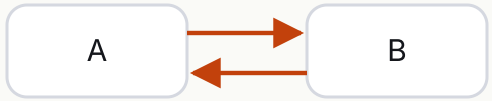
→

ARC
2011, clang: компилятор вставляет вызовы сам

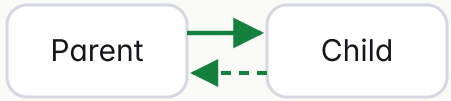
- Объекты — **в куче**, переменные — указатели (стек/регистры)
- Счётчик ссылок в каждом объекте; 0 → `dealloc`
- ARC — не сборщик мусора: работает при компиляции, детерминированно
- ARC не видит **циклов** — их разрывают `weak`-ссылки
- C-память (`malloc` / `free`) ARC не трогает

ARC: квалификаторы владения

Квалификатор	Владеет?	После уничтожения объекта	Когда
`__strong`	да (+1)	—	по умолчанию
`__weak`	нет	**обнуляется** (nil)	делегаты, обратные ссылки
`__unsafe_unretained`	нет	висячий указатель	редко, ради скорости
`__autoreleasing`	отложенно	—	выходной `NSError **`



strong ↔ strong: утечка



strong → / ← weak: освобождается

Блок, захвативший self, и self, хранящий блок, — тот же цикл: self → блок → self

weak разрывает цикл

10_memory.m

```
@autoreleasepool {
    Node *parent = [Node named:@"Parent"], *child = [Node named:@"Child"];
    parent.next = child;
    child.parent = parent;          // обратная ссылка — weak
}
NSLog(@"parent/child freed");
```

```
$ ./10_memory
dealloc Parent
dealloc Child
parent/child freed
```

> Правило: владеет тот, кто «сверху» (родитель, контроллер); обратные ссылки и делегаты — `weak`.

Пулы автоосвобождения

10_memory.m

```
for (int batch = 0; batch < 3; batch++) {
    @autoreleasepool {                  // временные объекты освобождаются каждую итерацию,
        for (int i = 0; i < 10000; i++) {      // а не в конце программы
            NSString *tmp = [NSString stringWithFormat:@"item-%d", i];
            (void)tmp;
        }
    }
}
NSLog(@"pool loop done");
```

```
$ ./10_memory
pool loop done
```

- `autorelease` = «release позже», при выходе из пула
- Вложенный `@autoreleasepool` в цикле снижает пик памяти

Итоги

- Конкурентность: `NSThread` и блокировки → GCD-очереди → `NSOperationQueue`
- ООП с одиночным наследованием, протоколы, категории, делегирование
- Блоки делают возможным функциональный стиль
- Сообщения вместо вызовов: `nil`, селекторы, KVC/KVO
- Виртуальной машины нет: машинный код + рантайм `libobjc` с `objc_msgSend`
- Память: подсчёт ссылок, ARC при компиляции, `weak` против циклов

Источники

1. Tutorialspoint. Objective-C Tutorial
   https://www.tutorialspoint.com/objective_c/index.htm
2. Feifan Zhou. Learn Objective-C in 24 Days (Binpress)
   https://www.binpress.com/learn-objective-c-24-days/
3. Apple. Objective-C Runtime Programming Guide
   https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ObjCRuntimeGuide/Introduction/Introduction.html
4. Apple. Concurrency Programming Guide
   https://developer.apple.com/library/archive/documentation/General/Conceptual/ConcurrencyProgrammingGuide/Introduction/Introduction.html
5. Apple. Key-Value Observing Programming Guide
   https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/KeyValueObserving/KeyValueObserving.html
6. Apple. Advanced Memory Management Programming Guide
   https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/MemoryMgmt/Articles/MemoryMgmt.html
7. Clang. Objective-C Automatic Reference Counting
   https://clang.llvm.org/docs/AutomaticReferenceCounting.html
8. GNUstep libobjc2 — рантайм для clang
   https://github.com/gnustep/libobjc2